



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

FastAPI - Introduzione

Angelo Sotgiu

angelo.sotgiu@unica.it

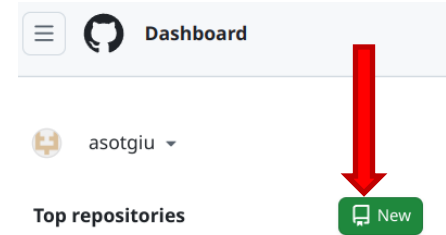
In questa lezione

- Creazione e configurazione di un repository GitHub
- Installazione e configurazione FastAPI
- Implementazione di un semplice server web
- Metodo GET
- Percorsi URL
- Parametri URL (query string)
- Parametri di percorso



Creazione di un repository GitHub

- Se non lo avete già fatto, create un account su github.com
- Per prima cosa, creiamo un nuovo repository dall'interfaccia web di GitHub
- Configuriamo le opzioni disponibili...
 - nome del repository
 - descrizione
 - visibilità (pubblico/privato)
 - aggiunta README, .gitignore e licenza
 - > scegliamo il .gitignore configurato automaticamente per Python
- ...e clicchiamo su "Create Repository"
- Il nostro repository inizialmente sarà vuoto (a parte eventuali README, .gitignore e licenza)



Clonazione del repository GitHub

- Cliccando su "Code", possiamo visualizzare e copiare il link del repository



- Apriamo un terminale dentro la cartella dove vogliamo clonare il repository, e digitiamo:

`git clone <link del repository>`

- Avremo adesso una cartella locale con lo stesso nome del repository, che sarà automaticamente collegata al repository remoto. Possiamo vederne lo stato e i branch disponibili con i comandi:

`git status`

`git branch`

Configurazione del repository GitHub

- Possiamo aprire il progetto da un'IDE (es. PyCharm, VS Code), ma basta anche un editor di testo...
- Per prima cosa, creiamo un nuovo branch chiamato *lab1* nel quale salvare il codice del primo laboratorio, e passiamo ad esso con checkout. Si può fare dall'IDE o da terminale con:

```
git branch lab1  
git checkout lab1  
(oppure in una sola riga git checkout -b lab1)
```

- Configuriamo adesso un file che specifica di che librerie ha bisogno il progetto, chiamato solitamente **requirements.txt**. Al suo interno aggiungiamo le seguenti librerie:

```
fastapi[standard]  
requests
```



Configurazione del repository GitHub

- Nella cartella del progetto, create anche un nuovo file **main.py**
- Quando creiamo un nuovo file, dobbiamo sempre dire a GitHub se tenerne traccia o meno:
 - se state usando un IDE, probabilmente vi verrà chiesto automaticamente
 - altrimenti da terminale usate questo comando:
git add <file>
se volete aggiungere tutti i file (usatelo con attenzione) -> git add .
- Aggiorniamo il repository remoto! (questa operazione andrebbe eseguita di frequente)

```
git commit -am "<commit_message>"  
git push
```

Creazione dell'environment

- Possiamo ora creare un nuovo environment Python (almeno 3.10 o superiore) utilizzando virtualenv o conda. Se uso conda da terminale:

```
conda create -n <nome_env> python=3.12  
conda activate <nome_env>
```

- Installiamo adesso i requisiti del progetto, sempre da terminale:

```
pip install -r requirements.txt
```

- Potremo aggiungere in seguito altri requisiti nel file requirements.txt

Perché FastAPI?

- Scritta in Python
- Facile da usare
- Molto efficiente
- Riduce tanti errori e bug
- Già usata in molte applicazioni reali
- Guida introduttiva: <https://fastapi.tiangolo.com/tutorial/>



Prima applicazione web

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

 oggetto che rappresenta l'istanza dell'applicazione

```
@app.get("/")
```

 decoratore che specifica il metodo HTTP e il percorso

```
def hello_world():
```

 funzione chiamata a seguito di una richiesta GET

```
    return "Hello World!"
```

 (chiamata path o endpoint function)

Proviamo a lanciare l'applicazione da terminale (dalla cartella dove si trova il file main):

```
uvicorn main:app
```

Prima applicazione web

Possiamo notare che:

- l'app viene eseguita sul **localhost**, ovvero l'indirizzo IP del vostro dispositivo...
- ...e sulla porta 8000 (si può cambiare passando il parametro **--port**)

Cliccate ora sul link <http://127.0.0.1:8000>, o digitate nel browser questo indirizzo: cosa succede?

- il vostro browser ha eseguito una richiesta HTTP GET, e il server ha risposto chiamando la funzione *hello_world* e restituendo l'output in una risposta HTTP
- la stringa di output è stata automaticamente codificata in formato JSON



Prima applicazione web

- Possiamo fare una richiesta tramite codice per ispezionare meglio cosa succede. Lasciate il terminale con l'applicazione web in esecuzione, ed eseguite questo codice python:

```
import requests
```

```
response = requests.get("http://localhost:8000")
```

- Abbiamo fatto esattamente la stessa cosa di prima, simulando un client.
- Adesso ispezioniamo la richiesta (accessibile tramite `response.request`) e la risposta!

Esecuzione dell'app con ricaricamento automatico

- Ci capiterà spesso di modificare il codice dell'applicazione web e volerla eseguire con le nuove modifiche. Per evitare di rilanciarla ogni volta, possiamo usare questo comando:

```
uvicorn main:app --reload
```

- Nelle ultime versioni di FastAPI è stato introdotto un nuovo comando:

```
fastapi dev main.py
```

- il reload è incluso di default
- non c'è bisogno di specificare l'app da eseguire (a meno che non abbiamo più di un'istanza nel nostro main!)

Percorsi URL

- Proviamo adesso a modificare il percorso URL al quale la nostra applicazione web è in grado di rispondere alle richieste GET:

```
@app.get("/hello")  
def hello_world():  
    return "Hello World!"
```

- Navigate ora all'indirizzo <http://127.0.0.1:8000/hello>
(non funziona? avete rilanciato l'app con reloading?)
- Cosa succede invece all'indirizzo root <http://127.0.0.1:8000/>?
PS: notate dal log nel terminale che il sistema gestisce automaticamente gli status code

Parametri URL (query string)

- Complichiamo un po' le cose: cerchiamo di gestire i parametri che possono essere passati tramite URL:

`http://<host>[:<port>][/<path>[?<query>]`

- Nello specifico, la query è composta da:
 - carattere ? dopo il path
 - coppia key=value per ogni parametro, separate da un carattere &
- Lato web server, ci basta definire i parametri della funzione che gestisce le richieste

```
@app.get("/")  
def endpoint_function(param1, param2, pippo, ...):  
    ...
```

Parametri URL (query string)

- Proviamo a passare alla nostra applicazione qualche parametro, cliccando su <http://127.0.0.1:8000?param1=1¶m2=2>
- Non succede niente, ma nel log sul terminale possiamo vedere come siano stati correttamente ricevuti
- Modifichiamo adesso la nostra funzione:

```
@app.get("/")  
def hello_world(param1, param2):  
    return f"Hello World! param1={param1} param2={param2}"
```

- Cliccando di nuovo il link sopra, possiamo vedere come i parametri siano ricevuti correttamente dal web server

Parametri URL (query string)

- Attenzione: il web server adesso si aspetta di ricevere esattamente quei due parametri! Queste richieste genereranno un errore:

<http://127.0.0.1:8000>

<http://127.0.0.1:8000/?param1=1>

<http://127.0.0.1:8000/?param1=1&pippo=2>

- E se volessimo dei parametri opzionali?
Basta definire i loro valori di default (oppure `None`) nella firma della funzione:

```
@app.get("/")  
def hello_world(param1, param2=2):  
    return f"Hello World! param1={param1} param2={param2}"
```


Typing

- Python non è un linguaggio tipizzato dinamicamente: il contenuto di una variabile può essere qualsiasi cosa
(a differenza di altri linguaggi come C e Java, dove dobbiamo esplicitamente dichiarare il tipo di dato che una variabile conterrà, es. *int i = 0*)
- Questo fornisce tanti vantaggi, ma può essere anche fonte di bug e vulnerabilità, e spesso rende il codice difficile da capire
- Dalla versione 3.5, Python ha introdotto il concetto di **type hints**: possiamo esplicitare per qualsiasi variabile il tipo di dato che *ci aspettiamo*.
- In realtà poi, a runtime, questo non viene fatto rispettare, ma è utile in fase di sviluppo.
A meno che non si usi qualche tool aggiuntivo che metta in atto queste regole, validando i dati: FastAPI usa Pydantic a questo scopo.

Typing

- Per usare i type hints, basta aggiungere dopo la dichiarazione di una variabile i due punti (:) e il tipo di dato richiesto. Al posto di:

```
name = "name"  
number = 0  
values = ["a", "b"]  
dictionary = {"a": 0}
```

avremo:

```
name: str = "name"           # stringa  
number: int = 0              # numero intero  
values: list[str] = ["a", "b"] # lista di stringhe  
dictionary: dict[str, int] = {"a": 0} # dizionario {stringa: intero}
```

Typing

- Questa funzionalità si usa soprattutto nelle firme delle funzioni, per specificare che tipo di parametri ci aspettiamo e che valore debba restituire la funzione:

```
def somma(x: int, y: int) -> int:  
    return x + y
```

```
def stampa(testo: str) -> None:  
    print(testo)
```

- Se i tipi di dati specificati non sono rispettati, il programma potrà comunque essere eseguito senza problemi, ma l'IDE mostrerà degli alert che possono essere molto utili in fase di sviluppo.
- Come già detto, FastAPI usa un tool che **obbliga** a rispettare i tipi di dati richiesti!

Typing per Parametri URL

- Per noi questa funzionalità è davvero utile: possiamo assicurarci che vengano passati i tipi di dati corretti senza grandi sforzi. Ad esempio, se volessimo solo numeri interi:

```
@app.get("/")
def hello_world(param1: int, param2: int = 2):
    return f"Hello World! param1={param1} param2={param2}"
```

- Adesso, cliccando su questo URL riceveremo un errore:
<http://127.0.0.1:8000/?param1=1¶m2=pippo>
- Vedremo più avanti altri vantaggi nell'utilizzare type hints!
Consiglio: usateli sempre, anche al di fuori di FastAPI...

Per maggiori dettagli: <https://fastapi.tiangolo.com/python-types/>



Parametri di percorso

- Possiamo definire degli URL *parametrici* e gestirli tutti all'interno della stessa funzione.

```
@app.get("/items/{item_id}")  
def hello_world(item_id):  
    return f"Hello World! item_id={item_id}"
```

- In questo caso, possiamo fare una richiesta all'indirizzo http://127.0.0.1:8000/items/{item_id} dove {item_id} può essere qualsiasi cosa, e sarà passato alla funzione come parametro

Parametri di percorso

- Possiamo utilizzare contemporaneamente parametri URL (query string) e di percorso, e applicare la validazione tramite type hint dove richiesto:

```
@app.get("/items/{item_id}")
def hello_world(item_id: int, param1: int, param2: int = 2):
    return f"Hello World! item_id={item_id}, param1={param1} param2={param2}"
```

- Adesso, cliccando su questo URL riceveremo un errore:
<http://127.0.0.1:8000/items/pippo?param1=1>
- L'ordine degli argomenti nella firma della funzione non conta, saranno recuperati tramite il loro nome! (fermo restando che Python richiede che i parametri opzionali vadano **sempre** dopo tutti quelli non-opzionali)



Parametri di percorso

- Attenzione: se vogliamo definire un endpoint parametrico, e un altro endpoint dove il parametro lo fissiamo noi, quest'ultimo andrà sempre dichiarato prima in un'altra funzione:

```
@app.get("/items/pippo")  
def hello_world():  
    return f"Hello World! - ENDPOINT PIPPO"
```

```
@app.get("/items/{item_id}")  
def hello_world(item_id: str):  
    return f"Hello World! item_id={item_id}"
```

- Altrimenti, verrebbe sempre chiamata la funzione parametrizzata...